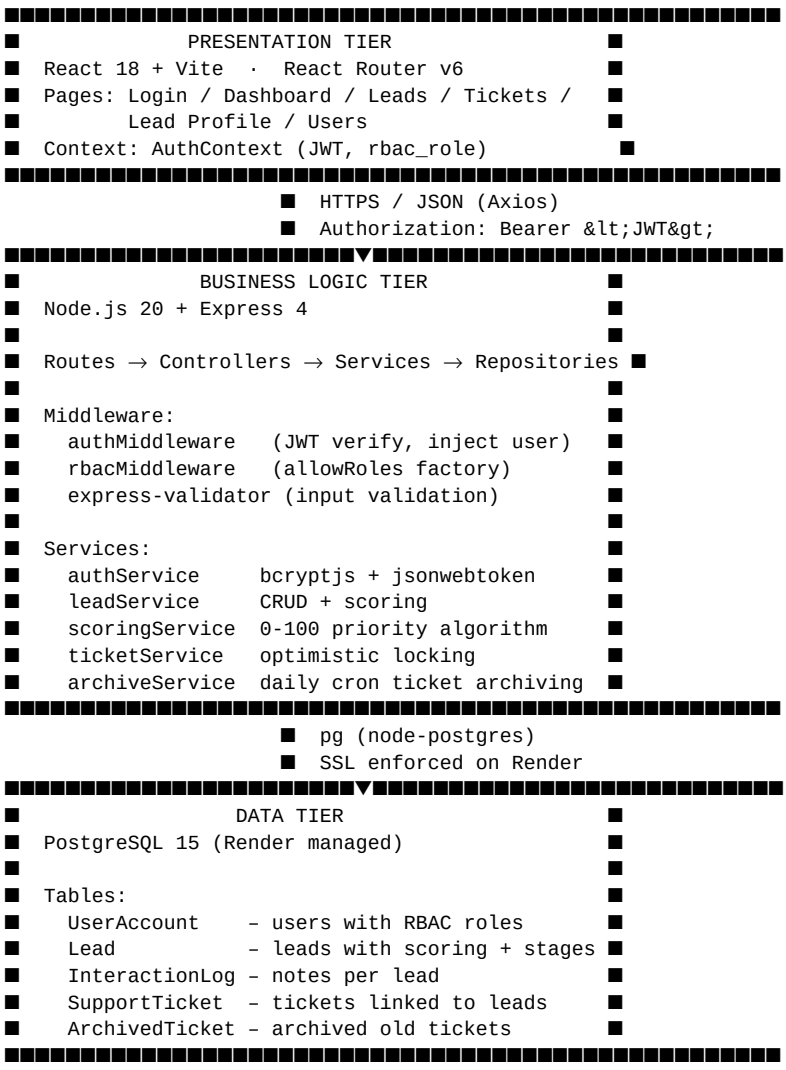
Architecture Description – WBOCRM

Overview

WBOCRM follows a classic **3-tier web application architecture** with a clear separation between presentation, business logic, and data layers. The system is designed for deployment on Render (cloud PaaS) with a PostgreSQL managed database.



Frontend Architecture

Layer	Technology	Purpose
Build	Vite 5	Fast HMR dev server, optimised production bundle
UI	React 18	Component-based SPA
Routing	React Router v6	Client-side routing with protected routes
Auth state	React Context API	JWT + user object available app-wide
Session security	Idle timeout (30 min)	Auto-logout on inactivity via activity listeners
Offline resilience	localStorage	Draft ticket caching with retry/dismiss UI
HTTP	Axios	Centralised API client with auth header injection

Deployment	Render Static Site	Served as pre-built static files with SPA rewrite

Route Protection

- `ProtectedLayout` — redirects to `/login` if no user in context
- `RoleRoute` — redirects to `/dashboard` if `user.rbac_role` not in allowed list

Backend Architecture

Request Lifecycle

```
HTTP Request
  → Express Router (/api/*)
    → authMiddleware (verify JWT, attach req.user)
      → rbacMiddleware (allowRoles check)
        → Controller (parse request, call service)
          → Service (business rules, orchestration)
            → Repository (parameterised SQL queries)
              → PostgreSQL pool
```

Controller → Service → Repository Pattern

Each domain (leads, tickets, users, auth) has three files:

- **Controller**: HTTP concerns only — parse body, call service, return status codes
- **Service**: Business logic — validation, scoring, conflict detection
- **Repository**: SQL only — all queries in one place, no business logic

Key Design Decisions

JWT Authentication

Tokens are signed with `HS256`, expire in 30 minutes, and carry `user_id` and `rbac_role`. The secret is injected via `JWT_SECRET` environment variable.

RBAC Middleware

`allowRoles(...roles)` returns an Express middleware that reads `req.user.rbac_role` and returns 403 if the role is not in the allowed list. Roles: `admin`, `sales`, `support`.

Lead Scoring Algorithm

Calculated at creation time. Five weighted metrics (max 100 pts):

- Calls completed: up to 25 pts (`calls / 20 × 25`)
- Meetings held: up to 25 pts (`meetings / 5 × 25`)
- Budget potential: up to 20 pts (`budget / 100 × 20`)
- Company size: 5 / 10 / 20 pts (small / medium / enterprise)
- Email opens: up to 10 pts (`emailOpens / 20 × 10`)

Optimistic Locking on Tickets

PUT `/api/tickets/:id` requires `updated_at` in the request body. If the DB record's `updated_at` differs, a `409 CONFLICT` is returned, preventing lost updates (NFR-ST-05).

Dashboard Auto-Refresh

`DashboardPage` polls `/api/dashboard` every 2 seconds via `setInterval` with cleanup on unmount (NFR-ST-12).

Database Schema

```
UserAccount    (user_id PK, user_email UNIQUE, user_password, rbac_role, full_name, created_at)
Lead           (lead_id PK, email UNIQUE, contact_name, priority_score, pipeline_stage,
                deal_value, campaign_id, calls, meetings, budget, company_size, email_opens,
```

```
                user_id FK→UserAccount, created_at)
InteractionLog (log_id PK, note_text, timestamp, lead_id FK→Lead, user_id FK→UserAccount)
SupportTicket  (ticket_id PK, description, status, priority_level, updated_at,
                lead_id FK→Lead, user_id FK→UserAccount, created_at)
ArchivedTicket (ticket_id PK, description, status, priority_level,
                lead_id, user_id, created_at, updated_at, archived_at)
```

Foreign Key Cascade Strategy

- Lead deleted → InteractionLog and SupportTicket rows deleted (ON DELETE CASCADE in schema)
- UserAccount deleted → Lead.user_id set to NULL (no orphan leads)

Data Archiving Strategy

`archiveService.js` manages automatic ticket lifecycle:

- **Daily cron job** runs via `setInterval(24h)` starting on backend boot
- Moves tickets with status `Resolved` or `Closed` and `updated_at` older than 365 days
- Uses a PostgreSQL transaction: INSERT into `ArchivedTicket` → DELETE from `SupportTicket`
- Admin can trigger archiving manually via `POST /api/tickets/archive`
- Response returns `{ archivedCount }` with the number of tickets moved

GDPR/KVKK Compliance

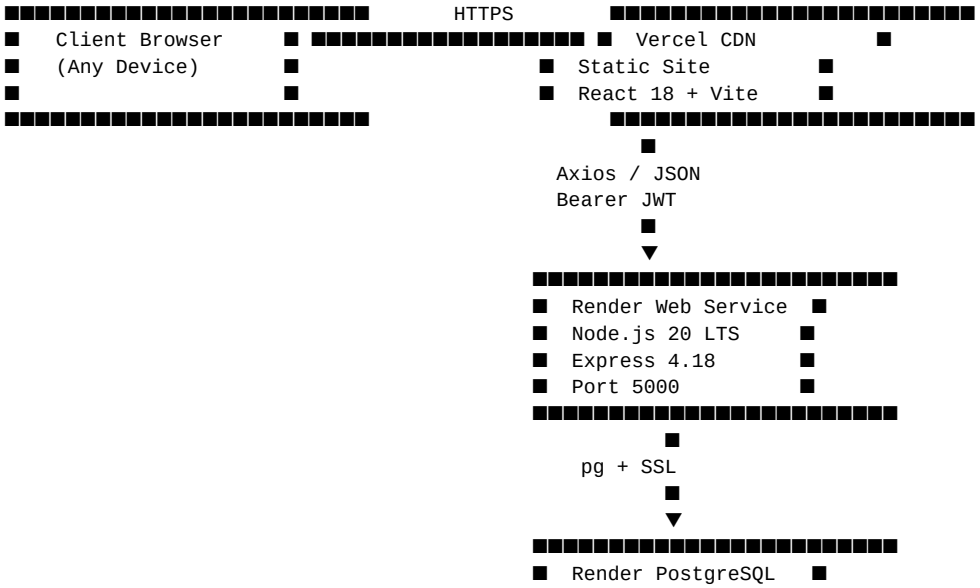
PII Masking (Data Minimisation)

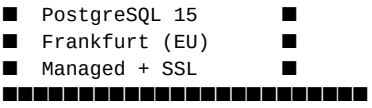
- Support-role users receive masked lead data: email as `a***@domain.com`, name as `A. L***`
- Masking applied in `leadController.js` before response, based on `req.user.rbac_role`
- Sales and admin roles see full unmasked data

Right-to-Erasure Endpoints

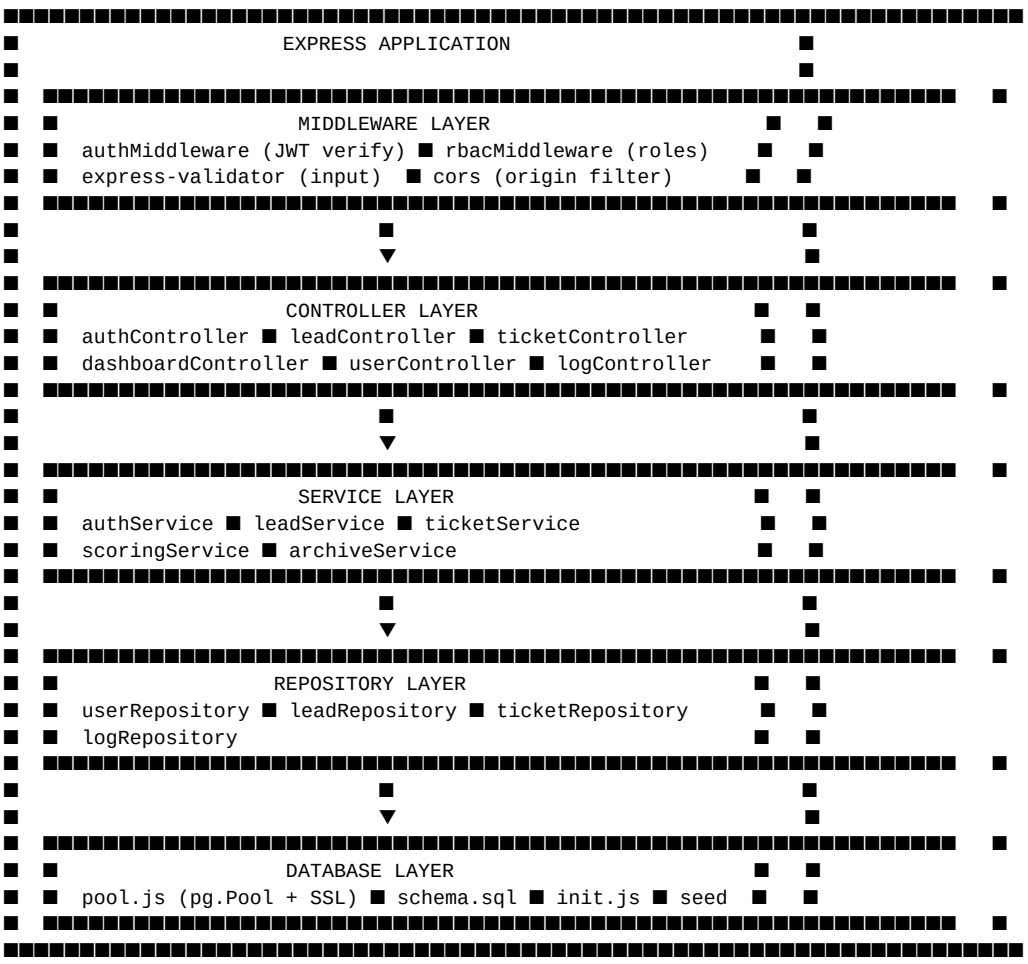
- `DELETE /api/leads/:id/personal-data` — anonymises lead email/name, deletes all interaction logs
- `DELETE /api/users/:id/personal-data` — anonymises user email/full_name
- Admin-only access; admin cannot erase their own account
- Last-admin guard prevents demoting the sole remaining admin (409 error)

UML Deployment Diagram





UML Component Diagram (Backend Layers)



Security Considerations

- Passwords hashed with bcryptjs, saltRounds=12
- JWT secret loaded from environment, never hard-coded
- All SQL uses parameterised queries (no string interpolation) — prevents SQL injection
- CORS restricted to `FRONTEND_URL` environment variable
- Input validation via `express-validator` on auth routes
- SSL enforced for all DB connections in production